EpiScope: A Big Data Platform for Disease Surveillance, Forecasting and Response Planning in Bangladesh

Group [TEAMMATE: number]: Shezan [ID], Tahsin Ahmed Rafi [ID], Rimjhim Dey [ID], Samira [ID]
CSE 4345 Big Data Analytics, Premier University, Chittagong
Repository: https://github.com/Shinzuu/episcope Video: [link added at v1.0]

***Abstract*—EpiScope turns routine health, climate and logistics data into district-level early warnings, one to four week forecasts, an allocation of a fixed national stock and a seasonal preparedness calendar. It is built as a Spark and Delta Lake medallion lakehouse over a synthetic encounter feed of more than 25 million records that takes its population, geography and weather from real public data. This draft fills the data engineering sections from the code and the evidence that exists on 20 October 2026; results marked [P] come from provisional test runs (SF=0.05 and SF=0.25) and will be replaced by the tagged v1.0 run.**

TEAMMATE: Samira (report lead)

Draft v0.1. Each yellow box names who writes or confirms that part. Every number must come from a file in `docs/experiments/` of the v1.0 run. Page limit: 15 pages plus appendices.

I. Introduction and Problem Analysis

Bangladesh carries a heavy and strongly seasonal load of infectious disease. The 2023 dengue season was the worst on record: WHO counted 308,167 cases and 1,598 deaths up to the end of November [1]. Data that could warn earlier already exists, but it sits in separate systems that differ in format, timeliness and quality, so most analysis happens after hospitals are already full.

The Ministry of Health (a teaching case) asks for a prototype that, for each of the 64 districts, (i) detects unusual rises in dengue, diarrhoea, influenza-like illness (ILI) and malaria, (ii) forecasts cases and hospital demand one to four weeks ahead, (iii) allocates a limited national stock, and (iv) gives a preparedness calendar for the next year. The users are not data scientists, the data is late and incomplete, no patient may be exposed, and the system must run on a 16 GB laptop.

TEAMMATE: Samira

Risk register (at least 5 risks): start from `docs/proposal.md` §6 and update each risk's status. Known updates: risk 2 (SF=1 within 45 minutes) waits on the scaling run; risk 3 (hidden drop) passed the drill; risk 7 (uneven commits) has happened.

II. CEP Attributes and Conflict Resolution

A. Attributes P1 to P7

- **P1 depth of knowledge.** Distributed processing and storage (Spark, Delta, partitioning, broadcast joins), keyed hashing built from `sha2`, surveillance statistics (EARS C2, CUSUM), count regression (NB-GLM), linear programming and visual encoding in D3.
- **P2 conflicting requirements.** The five conflicts below, each settled with a number.
- **P3 depth of analysis.** Thresholds come from a sweep against injected outbreaks, forecasts from a 52-origin backtest against a baseline, and α from a curve over budgets.
- **P4 unfamiliar issues.** Late records (MERGE on the latest report), a facility that goes silent and back-fills (onset-week counts, missing rather than zero), the same patient at two facilities (DQ-4), and the Dhaka skew.
- **P5 beyond standard codes.** ICD-10 three-character buckets, WHO IDSR [2] for the diseases and the weekly cadence, and our own suppression and allocation rules where no standard exists.
- **P6 stakeholders.** Table I.
- **P7 interdependence.** An error in bronze reaches the dashboard: counting by report date would create a false outbreak, which would raise forecast demand and move stock. Every gold row carries its bronze batch IDs, so such an error can be traced back.

TABLE I
Stakeholders and what the design does for each

Stakeholder	Need, and where the design meets it
MoH policymakers	Defensible allocation: LP against a population baseline, equity floor α
Epidemiologists	Transparent methods: EARS C2 and CUSUM, thresholds from a sweep, artefact handling
District officers	Early local warning: ranked alert table, district view
Hospital managers	Bed and supply demand with 80% intervals
Vector control	Hotspots by district and week; upazila withheld for privacy
Patients, public	Keyed hash, age bands, suppression of counts 1 to 4
Privacy regulator	Line-level data never leaves the lake; only aggregates reach the browser

B. Conflicts C1 to C5

C1 sensitivity against specificity. We allow at most two false alerts per district-year and take the most sensitive setting under that limit: EARS C2 at threshold 4.0 finds 97% of outbreaks with 1.27 false alerts per district-year[P] (Section VI).

C2 granularity against privacy. We publish district by week, not upazila. Every count from 1 to 4 is hidden, and a second cell is hidden in any row or column with only one hidden cell, so no total reveals it.

C3 efficiency against equity. The LP's equity floor α gives every district a share of each resource in proportion to its population.

TEAMMATE: Rafi

The recommended α and its cost from the curve, once the allocation has real need to allocate.

C4 freshness against cost. Forecasts and allocations are weekly decisions, so they run in a daily batch. Alerts can also run from a Structured Streaming file source that refreshes them as files land (Section IV); between files it holds almost no memory.

C5 accuracy against interpretability. The forecast is a negative-binomial GLM with seasonal terms and lagged rainfall and temperature: each coefficient can be read as a rate ratio. It beats seasonal naive with a relative WIS of 0.83[P].

TEAMMATE: Rafi

Confirm whether a more complex model was tried; if not, say so.

III. Architecture and Design Decisions

EpiScope follows the brief's reference architecture: a medallion lakehouse (bronze, silver, gold) on Apache Spark 4.0.1 [3] in local mode with Delta Lake 4.0.1 [4], analytics jobs that read gold and write gold, a serving step that exports suppressed JSON, and a D3.js v7 [5] dashboard. One command, `make all`, runs every stage in order (`reference`, `generate`, `bronze`, `silver`, `gold`, `analytics`, `serve`, `catalog`; Fig. 1); Java 17 and Python 3.12 come from `mise.toml` and every Python package is pinned in `requirements.txt`.

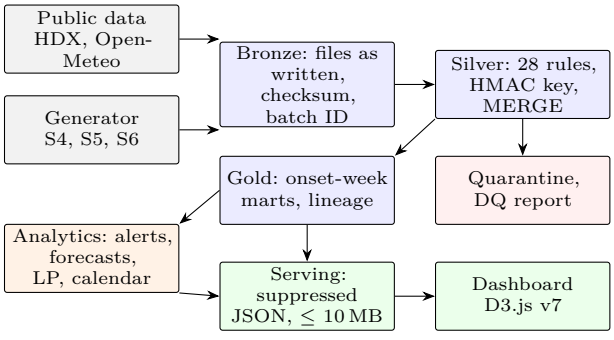


Fig. 1. EpiScope data flow. Blue: Delta lakehouse on Spark; one Makefile runs every stage.

Delta rather than plain Parquet. Late records (up to 21 days) must update their epi-week without duplicates. Delta's ACID MERGE makes that a single atomic statement, `replaceWhere` rewrites only the changed weeks of gold, and the transaction log gives time travel for "what did we know on date t". Iceberg would also work; Delta has the documented pairing with Spark 4.0.1 and pip packaging.

Local filesystem rather than HDFS or an object store (ING-5 not done). A Docker HDFS or S3 store costs 1–2 GB of RAM that Spark needs on the target laptop. All paths come from two environment variables, so the lake can move to another URI without code changes.

Partitioning. Bronze is partitioned by `batch_id`, because silver reads exactly one new batch at a time. Silver encounters and gold `cases_weekly` are partitioned by `epi_year` only: one year of gold holds at most $64 \times 4 \times 53$ rows, so a finer key (district or week) would create thousands of tiny files at SF=1 without saving any reads. The proposal planned `division` as a second key; we dropped it for the same reason.

Catalog. The jobs read and write by path, and a separate `make catalog` step registers every table as an external table (`bronze.encounters`, `gold.cases_weekly`, ...) in a Hive metastore stored inside the lake. Derby allows one process at a time, so keeping the catalog out of the jobs means a locked metastore can never stop the pipeline.

TEAMMATE: Samira

Check Fig. 1 and add the alternatives table (Airflow against a Makefile DAG, Iceberg against Delta, streaming against batch).

IV. Data Generation, Quality, Lineage and Privacy

A. Public sources and the generator

District boundaries, codes and the 2022 census population by age band and sex come from the HDX COD-AB and COD-PS datasets [6]; daily precipitation, temperature and humidity at each district centroid for 2021–2025 come from the Open-Meteo archive [7]. Open-Meteo allows about 5,000 weighted calls an hour and 64 districts over five years cost about 8,400, so the download is throttled and the processed reference files (0.6 MB) are committed. A clean clone therefore skips a two-hour download.

The generator (`generator/`) writes the three synthetic sources: encounters (S4) as one CSV folder per report date, a register of 736 facilities (S5) and daily stock snapshots (S6) as JSON. It is vectorised numpy, seeded per (seed, stream, district, year), so the same arguments give byte-identical output whatever the number of worker processes. Rates are per head of population, so the row count follows the census rather than a constant: SF=1 gives about 6.2 million encounters a year, or about 31 million for 2021–2025.

Each disease has its own annual cycle. Dengue peaks in early October, because OpenDengue's national counts

peak in October in 2022 and 2024, later than "follows the monsoon" suggests. Dengue responds to rainfall and temperature four to eight weeks earlier, diarrhoea spikes after extreme rainfall, malaria is concentrated in the three Hill Tracts districts and Cox's Bazar, and every count has negative-binomial noise. Reporting delay is a discretised negative binomial with mean 3 days truncated at 21, and 65% of Friday and Saturday reports move to Sunday (Bangladesh's weekend). The SF=0.05 run injected 35 outbreaks[P] of different shape and size, including a slow-growing one and a nationwide novel respiratory surge, and two reporting artefacts: a facility that goes silent for three weeks and then back-fills. They are written only to `ground_truth/`, which the detection code never reads (a test fails if it does).

The generator also plants the defects the quality rules must catch: 2% repeated `encounter_id`, 0.5% orphan `facility_id`, 0.5% invalid age, 0.5% invalid sex, 0.3% malformed ICD-10 codes and 0.3% onsets after the report date. A 14-day incremental drop (`make drop`) uses the same layout and carries records with onsets up to 21 days before the drop.

B. Bronze: keep what arrived

`pipeline/bronze/ingest.py` finds files by glob, never by name, date or count. It computes each file's SHA-256 and skips any file whose checksum is already in the ingest log, so a re-run reads nothing and a new drop loads only its own files. Every value of the contract fields is kept as a string, exactly as written, and each row gets the source, its file, the checksum, the batch ID and the load time (ING-1). One run is one batch; batch IDs start with a UTC timestamp, so they sort in load order.

C. Silver: rules, quarantine and de-identification

The rule catalogue (`pipeline/quality/rules.py`) has 14 rules for encounters and 7 each for facilities and stock (DQ-1). Each rule is a Spark column expression that is true when a row breaks it, with a stable reason code, a severity (quarantine or warn) and the requirement it serves. The six planted defects map one to one onto six rules:

- `DQ_DUP_ENCOUNTER` and `DQ_ORPHAN_FACILITY`;
- `DQ_INVALID_AGE` and `DQ_INVALID_SEX`;
- `DQ_ONSET_AFTER_REPORT` and `DQ_BAD_ICD10`.

The rest cover nulls, types, long delays, future dates and outcome values. Spark 4 runs in ANSI mode, where a plain cast of a bad string throws, so every rule reads values through `try_cast` and an unparseable value is a defect of its own. A failing row goes to `silver.quarantine` once, with the list of every code it broke. In the first SF=0.05 load, 66,979 of 1,681,896 encounter rows (4.0%)[P] were quarantined. Each load writes one row of rule pass rates to `gold.dq_report`, which the dashboard shows (DQ-2).

Before a clean row is stored, `patient_id` becomes an HMAC-SHA256 under a key held only in `.env` (PRV-1). Spark has no HMAC function and a Python UDF over 25 million rows is slow, so the HMAC is built from Spark's native `sha2` with the RFC 2104 inner and outer pads. A plain salted hash would not be enough: national IDs are few enough to enumerate, so the secret key is the only protection. The code refuses to run with a missing key or the placeholder from `.env.example`. Exact age becomes an age band.

D. Idempotent incremental loads (ING-4)

Clean rows are merged into `silver.encounters` with a Delta MERGE on `encounter_id`. A matched row is replaced only when the incoming `report_date` is later. That one condition gives both required properties: re-running a batch matches every row with an equal date and changes nothing, and a resubmitted or late record replaces the old copy and moves to its own onset week. The MERGE is limited to the epi-years in the batch (plus one either side), so partition pruning keeps it from scanning the whole table.

Silver records the weeks each batch changed: the onset weeks of new and replaced rows and the old weeks of replaced rows. Gold rebuilds only those weeks, plus the week after each, with a Delta `replaceWhere`.

E. Gold, lineage and patient de-duplication

`gold.cases_weekly` counts cases, admissions, deaths and lab confirmations per district, disease and ISO week of *onset*. Counting by onset rather than report date is what keeps the back-filled weeks of the silent facility from looking like an outbreak. Every district-week with at least one reporting facility gets a row for all four diseases, so a missing row means "nothing reported" rather than zero, which the detector uses to treat silent weeks as missing (AN-3). Each row carries the sorted bronze batch IDs of the records it counts, which link through the ingest log to file names and checksums (DQ-3).

Cross-facility patient de-duplication (DQ-4) flags a record as a repeat when the same patient key, disease, sex and compatible age band appear at another facility within seven days. On SF=0.05 it flagged 441 pairs, of which 440 were true repeats (precision 0.998)[P]. Recall within the window was only 0.11[P], because the generator draws age and sex afresh for a repeat visit, so most planted repeats fail the attribute check. We kept the strict rule: a false merge would hide a real case.

F. Time travel and the near-real-time path

Every gold build appends one row to `gold._snapshots`: the Delta version of the weekly case table it wrote and the latest report date that version contains. The `asof` module answers "what did we know on date t" by reading the newest version whose data ends on or before t with Delta's `versionAsOf` (ING-7). A test checks that the answer for a date between two loads leaves out a record that arrived 21 days late, which is exactly what a backtest origin should not see.

`pipeline/stream.py` (ING-6) is a Structured Streaming query over the encounter file source. Each micro-batch with new files runs the same incremental chain as the daily batch (bronze, silver MERGE, gold rebuild of the changed weeks, detection, export) in one Spark session, so a streamed record passes the same rules and keeps the same lineage. The bronze checksum log makes the stream and the batch safe to mix: whichever sees a file first loads it. We measured the time from the newest file of a 14-day drop to refreshed alerts twice at SF=1. With a 60 s trigger it was 301.5 s for 112 files (bronze 32 s, silver 118 s, gold 54 s, detection 32 s, export 22 s, the rest waiting for the trigger), 1.5 s over the 5-minute target. Since most of the miss was trigger wait, the default is now 30 s; the next drop of 80 files took 219.2 s (bronze 29 s, silver 96 s, gold 41 s, detection 25 s, export 18 s).

G. Hidden-drop drill

`scripts/hidden_drop_drill.sh` rehearses the instructor's test: clone, `make setup`, `make all`, a new 14-day drop with an unseen seed, the incremental steps, then the same steps again. On 21 October, on the current code at SF=0.05, it passed every check: the drop (seed 3006) became one new bronze batch, silver gained 8,741 rows, the data moved from 31 December 2025 to 14 January 2026, the re-run changed nothing and left zero duplicate IDs and zero duplicate gold keys, and the export was 9.78 MB with no count from 1 to 4. It took 38 minutes, including `make setup`.

V. Analytics Methods

TEAMMATE: Rafi: confirm this section, written from the code

Check each choice below against what you intended, and add the literature for each method.

A. Outbreak detection (AN-1, AN-3)

Both detectors run per district and disease on a series prepared in Spark (`analytics/detection/`). First, every week a district reported nothing at all is marked missing, as is any week where fewer than 80% of its expected facilities reported; above that, the count is scaled by 1/completeness. Second, counts are divided by a seasonal index per division, disease and week of year, taken as the median over the *other* years, so an outbreak cannot raise its own expected value.

EARS C2 [8] compares each week with the mean and standard deviation of lags 3 to 9 and runs on Spark window functions. CUSUM accumulates standardised excesses over a 26-week baseline behind a two-week guard band and runs in `applyInPandas`. Neither may alarm unless the last four weeks hold at least five cases, which keeps near-empty series quiet. Farrington [9] was ruled out: it needs about five years of history before its first test week, and we have five in total.

The reporting artefact is handled twice: gold counts by onset week, so back-filled records land on the weeks they happened, and the silent weeks are missing rather than zero.

B. Forecasting (AN-4)

For dengue and diarrhoea, one model per district and horizon $h = 1, \dots, 4$:

$$\log \mu_t = \beta_0 + \sum_{k=1}^{2} \text{Fourier}_k(t) + \beta_r \, \text{rain}_{t-\ell} + \beta_T \, \text{temp}_{t-\ell} + \beta_y \log(1 + \text{level}_{t-h}) \quad (1)$$

with lags ℓ of four to eight weeks and $\text{Var}(y) = \mu + \alpha\mu^2$ (NB2), fitted with statsmodels. Because every weather lag is at least as long as the horizon, no weather forecast is needed. Districts are fitted in parallel through a cogrouped `applyInPandas`. Predictive quantiles are the plug-in NB quantiles, which ignore parameter uncertainty.

C. Demand, allocation and calendar (AN-6, AN-7, AN-9)

Forecast cases become demand for beds, IV saline, NS1 kits, ORS and antimalarials through admission shares estimated from the data, lengths of stay and units per case or admission, all in `demand_assumptions.py` and marked as our assumptions. The LP plans against the 90th percentile.

The allocation is the brief's LP solved with PuLP 3.3.2 and CBC, with one change: unmet need for each resource is divided by its national net need, so a bed and a test kit are not worth the same in one objective. With $\alpha = 0$ the LP is pure efficiency; with $\alpha = 1$ every resource is split by population. The model is feasible for every α because $x = 0$ satisfies every constraint.

The preparedness calendar fits STL per division and disease in `applyInPandas`. Onset is the first week after the trough where the seasonal index passes 25% of the trough-to-peak range; supplies are pre-positioned three weeks before onset and campaigns start four to six weeks before it.

VI. Evaluation

The detection and backtest numbers come from the SF=0.25 build with real weather (seed 42) of 20 October[P], in the detection trade-off and backtest metric files under `docs/experiments`.

A. Detection

`make evaluate` scores every alert against the 35 injected outbreaks in `ground_truth/`, the only code allowed to read that folder. Consecutive alert weeks merge into one episode; an episode that overlaps no outbreak window counts as a false alert. The sweep picks, for each method, the most sensitive setting that keeps false alerts at or below two per district-year. Table II shows the chosen operating points.

EARS C2 is the better default here: it finds more outbreaks, sooner, with fewer false alerts. We expected CUSUM to do better on slow growth, but on the one slow outbreak both methods found it late, EARS C2 after 48 days and CUSUM after 69.

TABLE II
DETECTION AT THE CHOSEN OPERATING POINTS (SF=0.25)[P]

	EARS C2	CUSUM
Setting	threshold 4.0	$k = 0.5$, $h = 6$
Outbreaks detected	34 of 35	32 of 35
Sensitivity	0.97	0.91
False alerts per district-year	1.27	1.49
Positive predictive value	0.22	0.28
Median days to detection	8	9
Detected within 14 days	0.91	0.83

TEAMMATE: Samira (AN-2), Rafi (AN-3): confirm

Add the C1 trade-off figure from `detection_tradeoff.csv` (sensitivity against false alerts per district-year, one curve per method, the chosen points marked), results by outbreak shape, and the artefact facility with the naive detector and with ours. Replace with the v1.0 numbers.

B. Forecast backtest

The backtest refits the per-district NB-GLM at 52 rolling origins (every fourth origin, reused in between) and forecasts one to four weeks ahead using only data up to each origin. The seasonal-naive baseline is scored on exactly the same district, disease, origin and horizon cells. The weighted interval score [10] uses eleven central intervals plus the median; relative WIS below 1 means the model beats the baseline.

TABLE III
BACKTEST OVER 52 ORIGINS, MODEL AGAINST SEASONAL NAIVE (SF=0.25)[P]

Disease	h	MAE	MAE base	Cov. 80%	Rel. WIS
Dengue	1	39.4	42.5	0.83	0.88
	4	41.4	44.9	0.82	0.89
Diarrhoea	1	18.5	26.3	0.80	0.70
	4	19.3	26.4	0.80	0.74
All	1–4	29.5	35.0	0.81	0.83

The model beats the baseline at every horizon for both diseases, by more for diarrhoea (relative WIS 0.72) than for dengue (0.88). Its 80% intervals cover 81% of outcomes overall, against 78% for the baseline, so the bands mean what they say.

TEAMMATE: Samira (AN-5): confirm

Horizons 2 and 3 are in `backtest_metrics.csv`. Add the WIS decomposition (for dengue, most of the model's WIS is underprediction, which matters for stock planning). Replace with the v1.0 numbers.

C. Allocation against the population baseline

TEAMMATE: Rafi

In the SF=0.25 run the national net need is zero for beds, IV saline, NS1 kits and ORS, and 8.8 courses of antimalarials, because the generated stock exceeds the forecast demand. The LP's "50% of need met against 1% for the population baseline" is therefore about antimalarials only. Fix the demand or stock assumptions, then report share of need met and the worst-covered district at five budgets, and the efficiency and equity curve.

D. Scalability and Spark optimisations

`scripts/scaling.sh` builds each scale factor from a fresh clone and times every stage (Table IV). The laptop has an Intel Core i5-8350U (4 cores, 8 threads) and 15.4 GB of RAM; Spark runs as `local[4]` with a 5 GB driver. SF=1 finishes in 28.3 minutes, inside the 45-minute gate. Silver grows with the row count and is the slowest stage at SF=1; analytics stays near 7 to 8 minutes at every scale, because it works on weekly series per district, not on rows. A first SF=1 attempt failed in silver when Spark's spill filled the small RAM-backed `/tmp`; spill now goes to a disk folder.

TABLE IV
MINUTES PER STAGE FROM A FRESH CLONE

	SF=0.25	SF=0.5	SF=1
Encounter rows (M)	8.5	17.1	34.1
Generate	0.2	0.4	0.6
Bronze	1.9	2.8	4.3
Silver	3.8	6.7	11.1
Gold	1.8	2.3	3.5
Analytics	6.9	7.1	8.0
Serve	0.7	1.1	0.9
Total	15.4	20.4	28.3
Raw / silver (GB)	1.2 / 0.8	2.2 / 1.5	4.1 / 2.9

One optimisation is measured. The silver job evaluated the 14 encounter quality rules, including the duplicate-ID rule that sorts the whole load, once for every output (quarantine, its count, the quality report and the clean MERGE). It now evaluates them once and persists the result. At SF=0.25 silver's Spark time fell from 325 s to 214 s, and every silver table kept the same row count and row hash (`silver_optimisation.json`).

TEAMMATE: Samira (ENG-3)

Spark UI screenshots from the SF=1 event log (kept outside the repository; it holds the padded HMAC key), the bottleneck stage, and a second optimisation with before and after numbers (candidates: AQE skew handling or salting on the Dhaka load; backtest parallelism, 365 s at SF=1 on 4 of 8 threads).

VII. DASHBOARD DESIGN AND USABILITY

The dashboard (`dashboard/`) uses only D3.js v7 and the TopoJSON client, both vendored with subresource-integrity hashes. It loads only the aggregated JSON exports, which must stay under 10 MB: the export keeps the most recent weeks that fit. Five views are linked: a district choropleth of incidence per 100,000 with alert markers, a weekly time

series with a brush, forecast and bands, a seasonal heatmap, an allocation view with a what-if panel, and a ranked early-warning table. Suppressed cells are hatched and every view shows the data-freshness time.

TEAMMATE: Rimjhim

Design rationale (encodings, colour-blind-safe palette, why these aggregation levels), screenshots, and the usability test with at least two non-CS users: tasks, findings, changes made.

VIII. Ethical, Societal and Sustainability Considerations

Privacy by design. No real patient data is used; every encounter is synthetic and labelled so. Direct identifiers stop at bronze: silver and gold hold only the keyed hash and age bands, the key never enters Git, and every export hides counts from 1 to 4 with complementary suppression, so a total cannot reveal a hidden cell. One residual risk is known and written in the code: the padded HMAC key appears in Spark's query plan, so Spark event logs of the silver job must be treated as secret and kept outside the repository.

Record-level extracts and public release (PRV-4). `pipeline/privacy/kanon.py` measures k-anonymity [11] over the quasi-identifiers (age band, sex, district, onset week, disease, facility) of any record-level extract. k-anonymity does not stop a homogeneity attack, where everyone in a class shares one diagnosis, so a disease extract also needs l-diversity. For a public release, differential privacy [12] is the stronger tool: the Laplace mechanism adds noise with standard deviation $\sqrt{2}/\varepsilon$ to each count, about 1.4 cases at $\varepsilon = 1$. The privacy budget adds up over every published cell and week, and noisy counts can be negative and must be post-processed. We would use it for the public release only; ministry users keep exact counts with suppression.

TEAMMATE: Samira: confirm

Equity (what α protects and what it costs), misuse (stigma of a named district, alert fatigue), the limits of big-data signals [13], and the energy and hardware footprint.

IX. Limitations and Future Work

The data is synthetic, so detection and forecast scores measure how well the methods recover patterns we planted, not real-world accuracy. The storage layer runs on the local filesystem, not HDFS or an object store (ING-5). Patient de-duplication is precise but misses most repeats whose age or sex was recorded differently.

TEAMMATE: All

Add each area's main limitation and one concrete next step.

References

[1] World Health Organization, "Dengue: Bangladesh, disease outbreak news DON498," https://www.who.int/emergencies/disease-outbreak-news/item/2023-DON498, 2023.

[2] ——, *Technical Guidelines for Integrated Disease Surveillance and Response (IDSR)*, 3rd ed. WHO, 2019.

[3] M. Zaharia *et al.*, "Apache Spark: A unified engine for big data processing," *Communications of the ACM*, vol. 59, no. 11, 2016.

[4] M. Armbrust *et al.*, "Delta Lake: High-performance ACID table storage over cloud object stores," *Proceedings of the VLDB Endowment*, vol. 13, no. 12, 2020.

[5] M. Bostock, V. Ogievetsky, and J. Heer, "D3: Data-driven documents," *IEEE Transactions on Visualization and Computer Graphics*, vol. 17, no. 12, 2011.

[6] OCHA ROAP and Bangladesh Bureau of Statistics, "Bangladesh subnational administrative boundaries (COD-AB) and population statistics (COD-PS)," Humanitarian Data Exchange, https://data.humdata.org/dataset/cod-ab-bgd, 2025.

[7] P. Zippenfenig, "Open-Meteo.com weather API," Zenodo, https://open-meteo.com, 2023.

[8] L. Hutwagner, W. Thompson, G. M. Seeman, and T. Treadwell, "The bioterrorism preparedness and response Early Aberration Reporting System (EARS)," *Journal of Urban Health*, vol. 80, no. 1, 2003.

[9] C. P. Farrington, N. J. Andrews, A. D. Beale, and M. A. Catchpole, "A statistical algorithm for the early detection of outbreaks of infectious disease," *Journal of the Royal Statistical Society A*, vol. 159, no. 3, 1996.

[10] J. Bracher, E. L. Ray, T. Gneiting, and N. G. Reich, "Evaluating epidemic forecasts in an interval format," *PLoS Computational Biology*, vol. 17, no. 2, 2021.

[11] L. Sweeney, "k-anonymity: A model for protecting privacy," *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, vol. 10, no. 5, 2002.

[12] C. Dwork, F. McSherry, K. Nissim, and A. Smith, "Calibrating noise to sensitivity in private data analysis," in *Theory of Cryptography Conference (TCC)*, ser. LNCS, vol. 3876, 2006.

[13] D. Lazer, R. Kennedy, G. King, and A. Vespignani, "The parable of Google Flu: traps in big data analysis," *Science*, vol. 343, no. 6176, 2014.

Appendix A
Policy Brief for the Director

TEAMMATE: Samira: two pages, no jargon. A starting outline from the evidence so far

Current situation. A working prototype covers all 64 districts and four diseases, on synthetic data shaped by real population and weather. **Recommendations.** (1) Use the weekly alert ranking: in testing it caught 34 of 35 planted outbreaks, typically within eight days, with about one false alarm per district a year. (2) Plan dengue stock from the forecasts, which beat the "same week last year" rule. (3) Pre-position supplies three weeks before each division's seasonal onset in the calendar. **How to read the dashboard**, **confidence and limits** (synthetic data; allocation not yet validated), and **data protection** (no names or IDs leave the secure store; small counts are hidden). Replace every number with the v1.0 run.

Appendix B
Contribution Matrix

TEAMMATE: All, agreed and signed

Work areas from Appendix B of the brief, one column per member (L, S or none), agreed share σ_i (total 100), commits and lines from `git shortlog -sn`, signatures. Ask the instructor to confirm that the moderation factor uses $N = 4$.

Appendix C
Individual Reflections

TEAMMATE: Each member writes their own half page

Shezan, Tahsin Ahmed Rafi, Rimjhim Dey, Samira.